

Chapter 5: Solving Problems That Have Multiple Solution Characteristics

Nature-Inspired Optimisation for Delivery Problems (2022)

N. Urquhart

Natural Computing Series, Springer

2022

Chapter Overview

This chapter extends our optimisation toolkit from single-objective to **multi-objective problems**. In practice, a delivery planner rarely cares about just one thing: cost, time, emissions, and number of vehicles all matter simultaneously, and improving one often worsens another. The chapter introduces the concepts and algorithms needed to navigate these trade-offs.

Sections at a Glance

- 1 **5.1** Last Mile Deliveries — the business context and why multiple objectives arise naturally.
- 2 **5.2** Vehicle Routing with Time Windows (VRPTW) — adding time constraints to the VRP.
- 3 **5.3** A Case Study: FabFoods Home Delivery — cost model, representation, decoding, and implementation.
- 4 **5.4** Finding Trade-Offs Using Pareto Dominance — multi-objective evolutionary algorithm and results.
- 5 **5.5** Exploring Many-Dimensional Fronts — parallel coordinates for visualising and filtering high-D Pareto fronts.
- 6 **5.6** Conclusions.

5.1 Last Mile Deliveries — Context I

The **last mile** is the final step in the supply chain: the movement of a parcel or grocery order from a local depot or fulfilment centre directly to the consumer's home. The term “last mile” is used even though the actual distance may be many kilometres.

The on-demand economy has caused an explosive growth in business-to-consumer (B2C) deliveries. Data from the UK Office of National Statistics (ONS) shows that online purchases rose from **7.7%** of UK retail sales in July 2011 to **26%** in July 2021, reaching **36%** in December 2020. The vast majority of those purchases require a home delivery.

5.1 Last Mile Deliveries — Context II

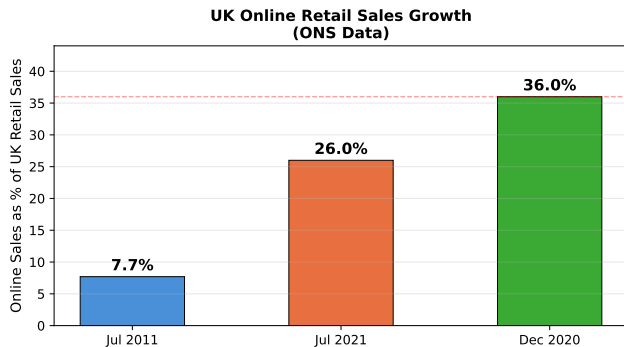


Figure: UK online retail sales as a percentage of total retail sales. Data from ONS (2021).

5.1 Last Mile Deliveries — Context III

Why Is This Hard? — The Four Objective Dimensions

When planning a typical last-mile delivery operation, the planner must balance at least four competing dimensions:

- **Financial** — capital costs (purchasing vehicles), running costs (fuel, maintenance), energy costs, and staff wages. These pull in opposite directions: fewer, longer routes cut vehicle overhead but increase driver wages.
- **Environmental** — CO₂ emissions and road congestion. Emissions depend on distance driven, load weight, and vehicle type (van vs cargo bike vs walking courier).
- **Customer Service** — customers specify *delivery time windows*; missing a window generates complaints or failed deliveries.
- **Delivery Technology** — the choice of vehicle type (vans, cargo bikes, walking couriers) affects cost, range, and capacity simultaneously.

These four dimensions interact in complex ways. For example, using more vehicles reduces individual route duration (good for time-window compliance) but raises the number of routes (bad for financial cost).

5.1 Last Mile Deliveries — Context IV

There is no single solution that is simultaneously best in every dimension — instead, there exist **trade-off solutions** that are *Pareto optimal*: you cannot improve one objective without worsening at least one other.

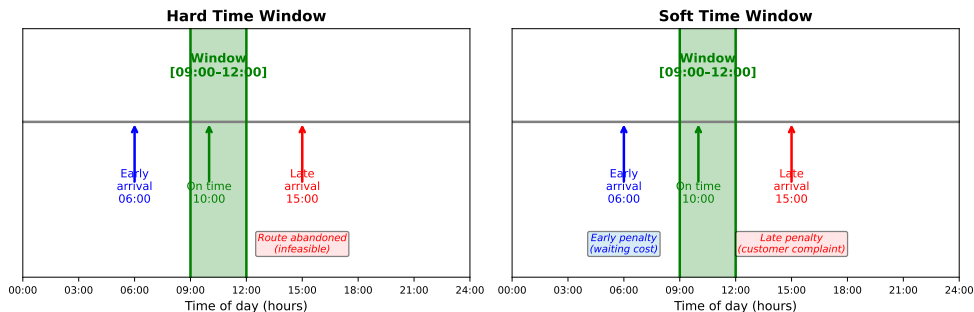
Key Insight

Because many real-world objectives conflict with each other, the correct output of a multi-objective solver is *not* a single answer but rather a **set of Pareto-optimal solutions** from which a human decision-maker can choose.

5.2 Vehicle Routing with Time Windows (VRPTW) I

The **Vehicle Routing Problem with Time Windows (VRPTW)** adds time constraints to the classic VRP: every customer i has a time window $[e_i, l_i]$ (where e_i is the *earliest* acceptable arrival time and l_i is the *latest*). Deliveries must be made within those windows, and each customer also requires a *service time* s — the time spent at the customer's door unloading goods.

Hard vs Soft Time Windows in VRPTW



5.2 Vehicle Routing with Time Windows (VRPTW) II

Figure: Hard time window (left) vs soft time window (right). In a hard window, a late arrival makes the route infeasible. In a soft window, a late arrival incurs a penalty but is still accepted.

5.2 Vehicle Routing with Time Windows (VRPTW) III

The VRP Family Tree

VRPTW is one member of a large family of vehicle routing variants:

- **CVRP** (Capacitated VRP) — each vehicle has a load capacity Q (Q).
- **VRPTW** (VRP with Time Windows) — hard or soft time-window constraints.
- **CVRPTW** (Capacitated + Time Windows) — the most commonly studied variant.
- **DVRP** (Dynamic VRP) — new orders arrive during the working day.
- **VRPPDTW** (VRP with Pickup and Delivery + Time Windows) — vehicles both collect and deliver.
- **VRPB** (VRP with Backhauls) — a subset of customers are collection points.

5.2 Vehicle Routing with Time Windows (VRPTW) IV

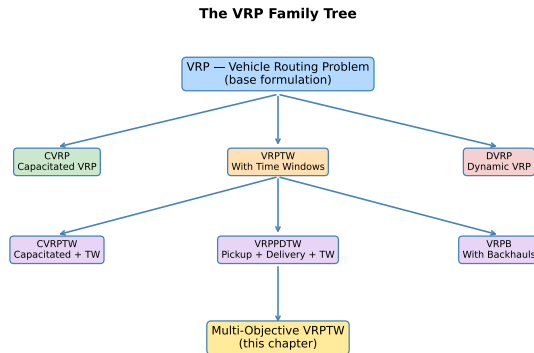


Figure: Hierarchy of VRP variants. The focus of this chapter is the multi-objective VRPTW at the bottom.

5.2 Vehicle Routing with Time Windows (VRPTW) V

Formal Definition of Hard and Soft Time Windows

Let vehicle v arrive at customer i at time a_i (a, for arrival).

- **Hard time window:** if $a_i < e_i$ (e , early bound), the vehicle *waits* at the customer's door until e_i — this is called *waiting time*. If $a_i > l_i$ (l , late bound), the route is declared **infeasible** and the decoder must start a fresh route for this customer.
- **Soft time window:** early or late arrivals are allowed but incur a **penalty cost** that is added to the objective function. The route remains feasible.

Worked Example: Waiting Time

Suppose a vehicle leaves the depot at 08:30 and the travel time to customer 5 is 8 minutes. The vehicle arrives at 08:38. However, customer 5's window is [12:00, 13:00]. The vehicle must **wait** at customer 5's door from 08:38 until 12:00 (a wait of 3 hours and 22 minutes). Service takes 5 minutes, so the vehicle **departs** customer 5 at 12:05. This waiting time counts toward the total route duration (and thus staff cost).

5.2 Vehicle Routing with Time Windows (VRPTW) VI

The service time s is typically a constant for all customers. In the FabFoods case study below, $s = 5$ minutes per customer.

Key Insight

Tighter time windows force more routes: if a customer's window closes before the vehicle can reach them from the previous stop, a new route must be started. This is why a 1-hour time window can roughly **double** the number of routes (and hence the cost) compared to a 14-hour window.

5.3 Case Study — FabFoods Home Delivery I

FabFoods is a fictional (but realistic) online grocer based on the **Augerat benchmark set**, a standard academic test suite for VRPTW problems. The problem uses *crates* as the unit of demand: each customer orders a certain number of crates of grocery items, and each vehicle holds a fixed number of crates. FabFoods wants to minimise **four objectives simultaneously**:

Four Objectives

- 1 **Distance** — total distance of all routes combined (in kilometres or route-distance units).
- 2 **Time** — total duration of all routes combined (in minutes), including travel, waiting, and service.
- 3 **Routes** — total number of routes, which determines how many vehicles are in use.
- 4 **Cost/Crate** — the financial cost per crate delivered (see the cost model below).

Why Four Objectives?

Distance minimisation saves fuel but may use many routes. Time minimisation reduces wage bills but may increase distance. Minimising routes reduces vehicle overhead but forces longer individual routes. Cost/Crate ties everything together into a single financial metric — but optimising it alone ignores environmental or customer-service impacts. Only a Pareto approach can show all the trade-offs.

5.3 Case Study — FabFoods Home Delivery II

The FabFoods Cost Model

The total cost of a delivery plan is computed in three steps:

$$vehicleCost = (noRoutes \times VEHFIXED) + (totalDistance \times VEHRUNNING)$$

Here, *noRoutes* is the number of routes (one vehicle per route); *VEHFIXED* = 164 (pounds, the fixed cost of putting one vehicle on the road for the day); *totalDistance* is the total kilometres driven; *VEHRUNNING* = 0.117 (pounds per kilometre, the per-km running cost).

$$staffCost = totalTime \times STAFF_RATE$$

Here, *totalTime* is the total route duration in hours; *STAFF_RATE* = 12 (pounds per hour per member of staff). One driver is assumed per vehicle, paid from departure to return.

$$solutionCost = vehicleCost + staffCost$$

$$costPerCrate = \frac{solutionCost}{totalDemand}$$

Here, *totalDemand* is the total number of crates to be delivered.

Python: VRPTW Decoder (Conceptual Implementation)

The core idea of the VRPTW decoder is straightforward to implement in Python. The following code demonstrates the logic used to translate a genome (ordered list of customers) into a set of routes while enforcing time windows and capacity constraints.

Listing 1: VRPTW Decoder in Python

```
1  SERVICE_TIME = 5      # minutes per customer visit
2  CAPACITY = 20         # maximum crates per vehicle
3  DEPOT_START = 8*60 + 30 # 08:30 in minutes since midnight
4
5  def decode_genome(genome, travel_times, windows, demands):
6      """
7      genome: list of (customer_id, new_route_flag) tuples
8      travel_times: dict[(from, to)] -> minutes
9      windows: dict[customer_id] -> (earliest, latest) in minutes
10     demands: dict[customer_id] -> crates
11     Returns: list of routes, each route = list of (customer, arrival_time)
12     """
13     routes = []
14     current_route = []
15     current_load = 0
16     current_time = DEPOT_START
17     current_pos = 'depot'
18
19     for customer, new_route_flag in genome:
20         travel = travel_times[(current_pos, customer)]
21         arrival = current_time + travel
```

Python: Single-Objective EA for VRPTW (from Book Listing 5.2)

The book implements the single-objective evolutionary algorithm for VRPTW in Java (Listing 5.2), but the logic translates directly to Python. The algorithm maintains a population and uses tournament selection to pick parents and a “worst” individual to replace.

Listing 2: Single-Objective EA Main Loop (Java → Python translation)

```
1  import random
2
3  def solve_single_objective(problem, objective_fn, evals_budget=10000,
4                             pop_size=50, tour_size=5, xo_rate=0.7):
5      """
6      Single-objective EA for VRPTW.
7      objective_fn: callable(individual) -> float (minimise)
8      Returns the best individual found.
9      """
10     population = [problem.random_individual() for _ in range(pop_size)]
11     for ind in population:
12         ind.evaluate(objective_fn)
13
14     best_so_far = min(population, key=lambda x: x.fitness)
15     budget = evals_budget
16
17     while budget > 0:
18         # Create child via crossover or mutation
19         if random.random() < xo_rate:
20             # Crossover: pick two parents by tournament
21             p1 = tournament_select(population, tour_size)
```


Python: Home Delivery Solver (Algorithm 13)

Algorithm 13 (Home Delivery Solver) orchestrates the full solution process. It runs the single-objective EA four times (once for each objective), runs the multi-objective EA ten times, pools all results, and extracts the grand Pareto front.

Listing 3: Home Delivery Solver — Python translation of Algorithm 13 / Listing 5.3

```
1  def home_delivery_solver(problem, n_tries=10):
2      """
3      Returns the grand non-dominated front (Pareto front) for the
4      FabFoods VRPTW problem.
5      """
6      solution_pool = []
7
8      # Phase 1: seed pool with single-objective optima (4 objectives)
9      for _ in range(n_tries):
10         for obj_fn in [cost_per_crate, total_distance,
11                        total_time, num_routes]:
12             best = solve_single_objective(problem, obj_fn)
13             solution_pool.append(best)
14
15     # Phase 2: run multi-objective EA many times
16     for _ in range(n_tries):
17         moea = NonDominatedEA(problem)
18         moea.solve()
19         solution_pool.extend(moea.get_non_dominated())
20
21     # Phase 3: extract grand front from entire pool
```

5.4 Finding Trade-Offs Using Pareto Dominance I

To find the set of all **non-dominated trade-off solutions** we use the concept of **Pareto dominance**, introduced informally in Chapter 4 and formalised here for multiple objectives.

Definition: Pareto Dominance (Minimisation)

Let $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_k(\mathbf{x}))$ denote the vector of k (k) objective values of solution $\mathbf{x}$. Solution $\mathbf{a}$ **dominates** solution $\mathbf{b}$ — written $\mathbf{a} \prec \mathbf{b}$ (prec, meaning “is better than”) — if and only if:

- ① $f_i(\mathbf{a}) \leq f_i(\mathbf{b})$ for *all* $i = 1, \dots, k$ ($\mathbf{a}$ is no worse on every objective), and
- ② $f_j(\mathbf{a}) < f_j(\mathbf{b})$ for *at least one* j ($\mathbf{a}$ is strictly better on at least one objective).

A solution is **Pareto optimal** (non-dominated) if no other feasible solution dominates it. The set of all Pareto-optimal solutions forms the **Pareto front**.

5.4 Finding Trade-Offs Using Pareto Dominance II

How to Read the Definition

Think of it this way: solution A dominates solution B if A is *at least as good as B in every category* and *strictly better in at least one category*. If A delivers groceries for less cost, takes less time, needs fewer vehicles, and travels less distance than B, then A dominates B and B would never be chosen. The Pareto front consists of all solutions where any improvement in one dimension *necessarily* comes at the cost of another.

5.4 Finding Trade-Offs Using Pareto Dominance III

START



Run single-obj EA
x4 objectives
(10 tries each)

40 solutions



Run multi-obj EA
(NonDepsEA)

non-dominated solutions

5.5 Exploring Many-Dimensional Fronts I

When the Pareto front contains many solutions across four dimensions, it is impossible to display it as a simple 2D scatter plot. We need a richer visualisation. The technique introduced in this section is the **Parallel Coordinates plot** (Inselberg 2009).

What Is a Parallel Coordinates Plot?

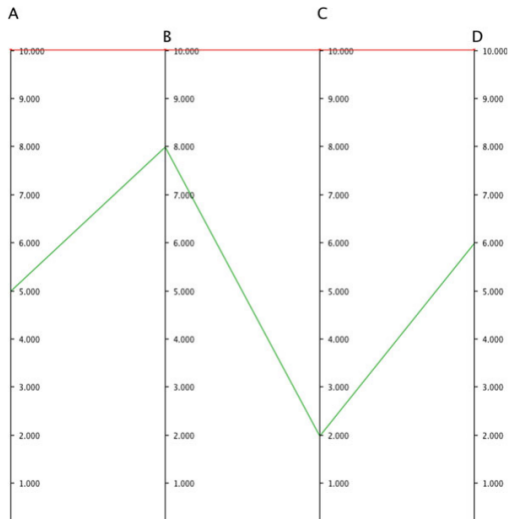
A parallel coordinates (PC) plot represents multi-dimensional data as follows:

- Draw one **vertical axis** for each dimension (objective). For our four-objective problem there are four axes: Distance, Time, Cost/Crate, and Routes.
- Each **axis is scaled** from its minimum to its maximum value across all solutions.
- Represent each **solution as a poly-line** — a broken line that crosses each axis at the value of that solution on that objective.
- A solution that is good on all objectives will be low on all axes (since we are minimising); a solution that is good on cost but poor on distance will be low on the cost axis and high on the distance axis.

5.5 Exploring Many-Dimensional Fronts II

108

5 Solving Problems That Have Multiple Solution Characteristics



Python: Parallel Coordinates Plot with Filtering

The following Python code reproduces the parallel coordinates plot from Figure 5.2 of the book and demonstrates how to apply filters programmatically.

Listing 4: Parallel Coordinates with Filters (Python/matplotlib)

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def parallel_coords_with_filter(solutions, axis_names,
5                                filters=None, highlight_color='blue'):
6      """
7      solutions: numpy array of shape (n_solutions, n_objectives)
8      axis_names: list of objective names
9      filters: list of (axis_index, min_val, max_val) tuples
10     """
11
12     n_sol, n_dim = solutions.shape
13     mins = solutions.min(axis=0)
14     maxs = solutions.max(axis=0)
15     norm = (solutions - mins) / (maxs - mins + 1e-9)  # normalise to [0,1]
16
17     fig, ax = plt.subplots(figsize=(10, 5))
18     x = np.arange(n_dim)
19
20     # Determine which solutions pass all filters
21     if filters:
22         passes = np.ones(n_sol, dtype=bool)
23         for (axis_idx, fmin, fmax) in filters:
```

5.6 Conclusions I

This chapter has demonstrated that real-world delivery problems are inherently multi-objective. The key contributions and takeaways are summarised below.

What Was Introduced

- 1 **The VRPTW formulation.** By adding time-window constraints to the VRP, the problem becomes significantly harder: tight windows can double the required number of routes and the associated cost. The decoder must handle waiting time, service time, and the possibility of route splitting when windows are missed.
- 2 **The FabFoods cost model.** A realistic four-component cost model (fixed vehicle cost, per-km running cost, staff time cost, and cost-per-crate) grounds the optimisation in real financial terms.
- 3 **The Home Delivery Solver (Algorithm 13).** A two-phase approach: seed a solution pool with single-objective optima (one for each of the four objectives), then run the multi-objective EA repeatedly and extract the non-dominated grand front.

5.6 Conclusions II

What Was Demonstrated

- **Why evolutionary algorithms suit multi-objective problems.** Because EAs maintain a *population*, they naturally produce many trade-off solutions in a single run — something impossible with single-solution methods like hill climbing.
- **The effect of time-window width.** The hypervolume results (Table 5.6) show that 14-hour windows produce fronts with $10\times$ – $100\times$ larger hypervolume than 1-hour windows, meaning a far richer set of trade-offs is available.
- **Parallel coordinates for decision support.** When a Pareto front has many solutions and many objectives, interactive filtering on a parallel coordinates plot allows a decision-maker to express preferences one constraint at a time and quickly narrow the choice to a practical shortlist.

5.6 Conclusions III

The Big Picture

Multi-objective optimisation does not replace human judgment — it enhances it. The algorithm's job is to find the *frontier of possibilities*; the decision-maker's job is to *choose a point on that frontier* based on priorities that may be difficult to quantify in advance. Parallel coordinates and similar visualisation tools are the bridge between algorithmic output and human decision-making.

5.6 Conclusions IV

Connections to the Rest of the Book

- **Chapter 2** introduced the VRP representation and the permutation-with-breaks encoding used here.
- **Chapter 4** introduced Pareto dominance for two objectives; this chapter generalises it to four.
- Future chapters build on the multi-objective framework to handle even more complex real-world scenarios.

5.6 Conclusions V

Further Reading

- Solomon, M.M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research* 35(2): 254–265.
- Inselberg, A. (2009). *Parallel Coordinates: Visual Multidimensional Geometry and Its Applications*. Springer.
- Augerat, P. (1995). *Approche polyédrale du problème de tournées de véhicules*. Ph.D. Thesis, Grenoble INP.
- de Rchefort, G. (2020). XDat application for parallel coordinates visualisation.
<https://www.xdat.org/>

Summary: Chapter 5 at a Glance

Key Concepts

- **Last mile:** final delivery to consumer home; multi-objective by nature.
- **VRPTW:** VRP + time windows; hard window $\Rightarrow$ infeasible if late; soft window $\Rightarrow$ penalty.
- **Pareto dominance:** A dominates B if A is no worse on all objectives and strictly better on at least one.
- **Grand Pareto front:** union of all non-dominated solutions from both single- and multi-objective runs.
- **Hypervolume:** area/volume of objective space dominated by the front; larger = better front.
- **Parallel coordinates:** visualise n -D fronts as parallel lines; filter over to find trade-offs.

Key Formulas

Vehicle cost: $\text{noRoutes} \times 164 + \text{dist} \times 0.117$

Staff cost: $\text{totalTime}(\text{hrs}) \times 12$

Cost per crate: $\frac{\text{vehicleCost} + \text{staffCost}}{\text{totalDemand}}$

Pareto dominance ($\mathbf{a} \prec \mathbf{b}$): $f_i(\mathbf{a}) \leq f_i(\mathbf{b}) \forall i$ and $\exists j : f_j(\mathbf{a}) < f_j(\mathbf{b})$

Algorithm 13 flow:

Single-obj $\times 4$ (10 tries each) $\rightarrow$ MOEA (10 tries) $\rightarrow$ extractNonDom $\rightarrow$ Grand Front